# TechPad Firmware: How It Works

An explanation of the firmware before any of it is written. Every pin and wire here comes from your routed board file (pcb.kicad_pcb, 18 Oct 2026). The key, layer and colour choices in section 6 are my defaults, and you can change any of them. Nothing has run on real hardware yet. 19 Oct 2026.

Confidence tags: **Certain** checked against your PCB file or official documentation. **Likely** standard practice, not yet tested on your board. **Guessing** a reasonable choice that may need changing once the parts arrive.

## 1. What firmware is

The XIAO RP2040 is a tiny computer. Firmware is the program stored on it. It does the same 5 jobs about a thousand times a second:

1. **Scan the keys** to see which of the 13 switches are pressed (12 keys plus the knob click).
2. **Read the knob** to see if it turned and which way.
3. **Send key presses to the computer over USB.** The computer sees TechPad as an ordinary keyboard, so it works on Windows, Mac and Linux without drivers. **Certain**
4. **Draw the OLED screen**: the layer name and the last key you pressed.
5. **Light the 12 RGB LEDs.** Keys glow when pressed.

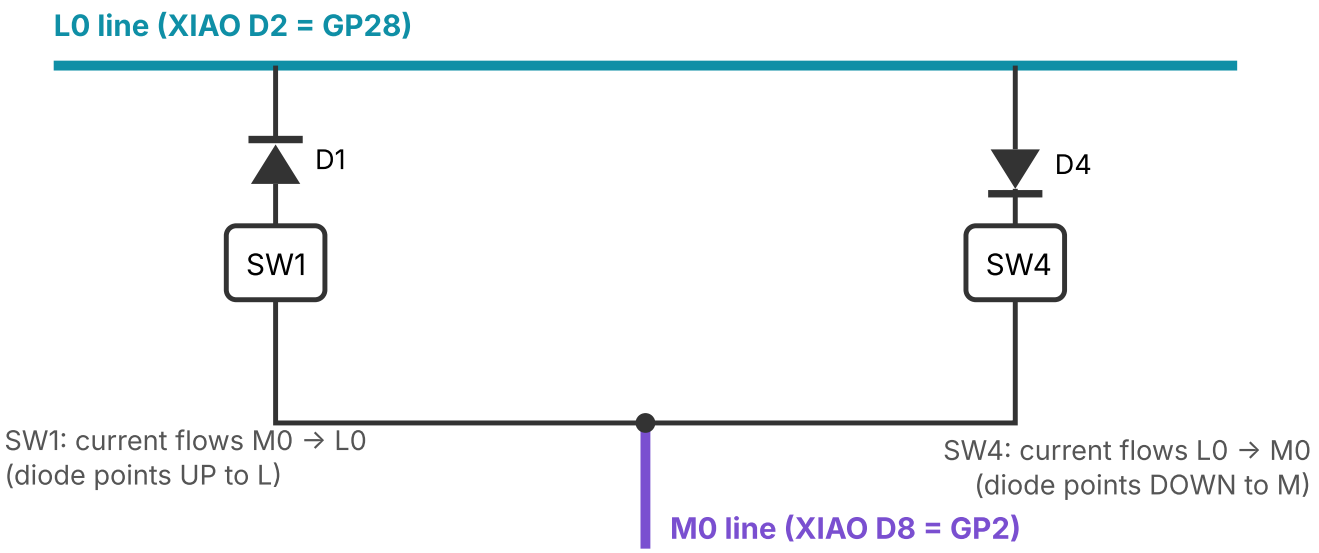
## 2. Why QMK

We're using **QMK**, the open-source firmware most custom keyboards run. You don't write it from nothing. QMK already handles USB, debouncing (filtering switch bounce), the knob, the OLED, the RGB effects and saving settings. **Certain** We add one small TechPad-specific piece (section 3) and the settings files. QMK was picked in the planning doc (Documentation/feasibility.md, section 4) for three reasons:

- It supports the RP2040 and the XIAO's pinout. **Certain**
- It has a "custom matrix lite" mode for wiring like yours. **Certain**
- It includes **VIA**, which lets an app change your keys over USB without reflashing. That's the hook your desktop app will use later. **Certain**

## 3. The tricky part: your duplex key matrix

The XIAO has only 11 usable pins. The knob takes 2, the OLED 2 and the LEDs 1, which leaves **6 pins for 13 switches**. A normal grid of rows and columns would need 7. Your board uses a **duplex matrix** instead: 3 **L lines** and 3 **M lines**, and up to **two** switches at each crossing, with their diodes pointing in **opposite directions**. **Certain**




One real crossing on your board: SW1 and SW4 both join L0 and M0, but their diodes point opposite ways.

A diode lets current through only one way, so the firmware can tell SW1 and SW4 apart by checking each direction separately. Every scan has **two passes**:

**Pass A: drive the L lines**
Pull one L line LOW and read the 3 M lines. A pressed key whose diode points toward L pulls its M line LOW too. Repeat for L0, L1, L2. **Finds 9 keys.**

**Pass B: drive the M lines**
Pull one M line LOW and read the 3 L lines. This finds the keys whose diode points toward M. Repeat for M0 and M1 (M2 has no keys in this direction). **Finds SW4, SW8, SW12 and the knob click.**

Lines that aren't being driven are set to "input with pull-up", so they float HIGH and don't interfere. After each line is pulled LOW, the firmware waits about 30 µs before reading so the line settles. Likely One full scan takes about 0.2 ms, far faster than a finger. Likely

This is the only TechPad-specific code, about 60 lines in `matrix.c`. QMK expects a grid, so the code reports the 13 switches as a 3 × 5 grid: **row = which L line**, **column 0–2 = M line from pass A**, **column 3–4 = M0/M1 from pass B**.

| Switch | Where it is | Current direction (from your PCB) | Pass | QMK position [row, col] |
|---|---|---|---|---|
| SW1 | top row, 1st | M0 → L0 | A | [0, 0] |
| SW2 | top row, 2nd | M1 → L0 | A | [0, 1] |
| SW3 | top row, 3rd | M2 → L0 | A | [0, 2] |
| SW4 | top row, 4th | L0 → M0 | B | [0, 3] |
| SW5 | middle row, 1st | M0 → L1 | A | [1, 0] |
| SW6 | middle row, 2nd | M1 → L1 | A | [1, 1] |
| SW7 | middle row, 3rd | M2 → L1 | A | [1, 2] |
| SW8 | middle row, 4th | L1 → M0 | B | [1, 3] |
| SW9 | bottom row, 1st | M0 → L2 | A | [2, 0] |
| SW10 | bottom row, 2nd | M1 → L2 | A | [2, 1] |

| | | | | |
|---|---|---|---|---|
| SW11 | bottom row, 3rd | M2 → L2 | A | [2, 2] |
| SW12 | bottom row, 4th | L2 → M0 | B | [2, 3] |
| SW13 | knob push | L0 → M1 | B | [0, 4] |

**If a key types the wrong thing after you solder**, its diode is probably in backwards. That moves the switch into the other pass, so it shows up as a different key. Likely Section 8 explains how to spot it.

## 4. Pin map

The firmware uses the RP2040 chip's own pin names (GP numbers), not the D numbers printed on the XIAO. Certain (Seeed XIAO RP2040 pinout.)

| Net on your PCB | XIAO pin | RP2040 GPIO | Used for |
|---|---|---|---|
| ENC_A | D0 | GP26 | Knob rotation A |
| ENC_B | D1 | GP27 | Knob rotation B |
| L0 / L1 / L2 | D2 / D3 / D6 | GP28 / GP29 / GP0 | Key matrix L lines |
| M0 / M1 / M2 | D8 / D9 / D10 | GP2 / GP4 / GP3 | Key matrix M lines |
| SDA / SCL | D4 / D5 | GP6 / GP7 | OLED (I²C bus 1) |
| DIN | D7 | GP1 | RGB LED data (12 LEDs) |

## 5. Knob, OLED and lights

### Knob (EC11)

Turning the knob makes contacts A and B click on and off slightly out of step. Which one changes first gives the direction. QMK decodes this itself. Certain One detent = one step, which can be changed later if it skips or doubles. Guessing The knob click is just switch 13 in the matrix.

### OLED (0.91", 128 × 32 pixels, SSD1306 chip)

The screen fits 4 lines of 21 characters. Certain Earlier we found that at a normal desk angle the top-row keycaps hide everything except the **top line**, so the top line carries what matters most: the layer and the last key you pressed. The other lines are visible when you look down from above. Likely

```
DEV       > Copy
Knob: Volume
RGB 40%   Reactive
</> TECHPAD
```

Mock-up of the screen. The top line updates on every key press.

The screen switches off after 60 seconds idle to prevent burn-in, and the next key press or knob turn wakes it. Likely Its address on the bus is 0x3C, the standard for these modules. Guessing It could be 0x3D, which is a one-line change if the screen stays blank.

### RGB (12 × SK6812 MINI-E)

- **Brightness:** capped at about 40% (around 290 mA worst case) to stay well inside USB power. Likely

- **Default look:** a dim colour for the current layer, and each key flashes bright when pressed, then fades (QMK's "reactive" effect). Certain
- **LED order:** the LEDs are wired as a snake (top row left to right, middle row right to left, bottom row left to right; checked on your PCB Certain). The firmware has a table that maps each LED to its key, so the right LED lights under the key you press.

## 6. Default keys (my proposal, change anything)

TechPad has 3 **layers**. A layer is a whole set of 12 key meanings. The bottom-right key always switches to the next layer (Dev → Media → Lights → Dev). The OLED shows the layer name and the keys glow in that layer's colour, so you always know where you are.

### Layer 1: DEV (cyan)

| | | | |
|---|---|---|---|
| **Copy** Ctrl+C | **Paste** Ctrl+V | **Cut** Ctrl+X | **Undo** Ctrl+Z |
| **Save** Ctrl+S | **Find** Ctrl+F | **Comment** Ctrl+/ | **Commands** Ctrl+Shift+P |
| **Terminal** Ctrl+` | **Switch app** Alt+Tab | **Lock PC** Win+L | **Next layer** → Media |

Knob: volume up/down. Click: mute.

### Layer 2: MEDIA (purple)

| | | | |
|---|---|---|---|
| **Previous** track | **Play / Pause** | **Next** track | **Mute** |
| **Vol –** | **Vol +** | **Screen –** brightness | **Screen +** brightness |
| **Calculator** | **Browser** home | **Files** Win+E | **Next layer** → Lights |

Knob: volume. Click: play/pause.

### Layer 3: LIGHTS (amber)

| | | | |
|---|---|---|---|
| **RGB on/off** | **Next effect** | **Bright –** | **Bright +** |
| **Colour –** hue | **Colour +** hue | **Speed –** effects | **Speed +** effects |
| **Flash mode** hold 1 s | | | **Next layer** → Dev |

Knob: LED brightness. Click: lights on/off.

"Commands" (Ctrl+Shift+P) and "Comment" (Ctrl+/) are VS Code shortcuts, which suit the developer look. Screen-brightness keys work on most laptops but not on desktop monitors. Likely

## 7. Getting new firmware onto the XIAO

**Why there's a "Flash mode" key:** once the case is closed, the XIAO's BOOT button is inside the tray, on the back of the PCB, and you can't press it. Certain So the firmware gives you two other ways in:

1. Hold **Flash mode** on the Lights layer for 1 second.
2. Hold the **top-left key** while plugging in the USB cable (QMK "Bootmagic"). This works even if a bad flash broke everything else. Certain

Either way, a USB drive called **RPI-RP2** appears. Drag `techpad.uf2` onto it, and the pad restarts running the new firmware. Certain You don't need any tools.

Once VIA is set up, you can change keys **without reflashing**: the VIA website or app (and later your own desktop app) rewrites the keys over USB, and they're saved on the XIAO. Certain

## 8. What I will write, and what's untested

| File (in the repo, Firmware/qmk/techpad/) | What it does |
|---|---|
| keyboard.json | Board description: USB name, matrix size, knob pins, the LED-to-key table and enabled features. |
| config.h | OLED bus pins, RGB brightness cap, bootmagic key. |
| matrix.c | The duplex scanner from section 3, the only hand-written hardware code. |
| halconf.h / mcuconf.h | Turn on the RP2040's I²C bus 1 for the OLED. |
| keymaps/default/keymap.c | The 3 layers, knob actions, OLED screen and layer colours. |
| keymaps/via/ | The same keys with VIA turned on, for live remapping. |
| techpad.uf2 | The compiled file you drag onto the XIAO. |

Before I send it, I'll compile it with the official QMK tools so it's free of build errors. Certain What can't be tested until the parts arrive:

- Whether every key maps to the right position. This depends on the diodes being soldered the right way round (section 3).
- The knob's direction and steps per click. If it runs backwards, that's a one-line swap. Guessing
- Whether the OLED address is 0x3C (section 5). Guessing
- The LED data signal: the RP2040 sends 3.3 V but the LEDs run on 5 V. It works on most builds, but sits at the edge of the LED's spec. If the first LED flickers, there's a known fix in 05_After_Parts_Arrive. Likely

Your README will say this firmware has been compiled but not yet tested on hardware, which is what Hack Club expects for a design-first submission.

**What I need from you:** say **"go"** to have me write and compile it with the defaults above. Or tell me what to change first, for example different keys, layer names, colours or knob actions.